# MANUAL TÉCNICO

**PROYECTO:** ResiPark - Sistema de Control de Accesos y Estacionamiento

**CURSO:** Introducción a la Programación y Computación 1

**SEMESTRE:** Segundo Semestre 2026

## 1. Introducción y Arquitectura del Sistema

ResiPark es un sistema desktop desarrollado en Java (Apache NetBeans) utilizando la librería Swing para la interfaz gráfica. La aplicación simula el control de entradas, salidas y asignación de parqueos en un residencial mediante un enfoque orientado a objetos y programación concurrente con hilos (multithreading).

El proyecto está diseñado bajo una arquitectura de capas bien definidas:

- **modelo**: Contiene los objetos del dominio (Residente, Vehiculo, EspacioParqueo, Evento).
- **estructuras**: Implementaciones manuales de estructuras de datos dinámicas con nodos propios, sin el uso del java.util Collections Framework ni arreglos estáticos.
- **hilos**: Contiene la lógica de concurrencia y sincronización (GaritaEntrada, GaritaSalida).
- **persistencia**: Encargado de la lectura y escritura en archivos de texto plano (GestorArchivos).
- **vista**: Interfaz gráfica de usuario desarrollada mediante código Java en Swing (VentanaPrincipal).

## 2. Descripción de Estructuras de Datos Propias

## 2.1. Lista Doblemente Enlazada (ListaDobleResidentes)

- **Uso:** Almacena los residentes registrados en el residencial.
- **Propiedades:** Cada nodo posee referencias a residente, anterior y siguiente.
- **Operaciones implementadas:**
    - insertar(Residente r): Inserta un nodo al final de la lista validando ID único.
    - buscarPorId(String id): Recorre los nodos desde la cabeza (cabeza) comparando IDs.
    - actualizar(id, nombre, casa, esSocio): Modifica los datos. Si se intenta cambiar el tipo de socio, valida previamente que sus vehículos tengan estado FUERA.

    - eliminar(String id): Elimina en cascada al residente y sus vehículos, únicamente si todos están en estado FUERA.
    - existePlacaGlobal(String placa): Recorre los residentes y sus listas simples para verificar que no existan placas duplicadas en el sistema.

## 2.2. Lista Simplemente Enlazada (ListaSimpleVehiculos)

- **Uso:** Almacena los vehículos asignados a cada residente.
- **Propiedades:** Cada nodo posee referencias a vehiculo y siguiente. Limita a un máximo de 3 vehículos por residente.
- **Operaciones implementadas:** insertar, buscarPorPlaca, eliminar (validando estado FUERA) y getCabeza() para iterar nodos.

## 2.3. Lista Circular Simplemente Enlazada (ListaCircularParqueo)

- **Uso:** Modela las áreas de estacionamiento (Área Socios A-C con 75 casillas, Área General E-I con 75 casillas).
- **Propiedades:** El último nodo apunta al primer nodo. Mantiene el puntero ultimoAsignado.
- **Algoritmo de Asignación Rotativa:**
    - Al buscar un espacio libre, comienza la búsqueda a partir de ultimoAsignado.siguiente.
    - Realiza una sola vuelta completa (recorrido circular). Si encuentra un espacio desocupado, lo asigna, actualiza la referencia de ultimoAsignado al nodo actual y retorna el espacio.

## 2.4. Cola FIFO (ColaFIFO)

- **Uso:** Gestiona los vehículos que esperan turno en la garita de entrada y solicitudes de salida.
- **Sincronización:** Métodos encolar(Vehiculo v) y desencolar() declarados como synchronized. Implementa wait() cuando la cola está vacía y notifyAll() cuando ingresa un nuevo vehículo para despertar los hilos dormidos.

## 2.5. Pila LIFO (PilaEventos)

- **Uso:** Historial de eventos y bitácora de auditoría del sistema.
- **Comportamiento:** Inserta nuevos eventos en el tope (tope). Al renderizar en pantalla, muestra los eventos del más reciente al más antiguo.

## 3. Concurrencia y Sincronización (Hilos)

El sistema utiliza tres hilos independientes que ejecutan en segundo plano:

1. **Garita Entrada 1 (GaritaEntrada):** Atiende concurrentemente la ColaFIFO de entrada.
2. **Garita Entrada 2 (GaritaEntrada):** Atiende en paralelo la misma cola de entrada.
3. **Garita Salida (GaritaSalida):** Procesa de forma concurrente las solicitudes de salida.

- **Sincronización:** La concurrencia entre Garita 1 y Garita 2 sobre la cola de entrada y sobre las listas circulares de parqueo se gestiona con bloques synchronized. Esto evita condiciones de carrera (*race conditions*) y previene que dos garitas asignen la misma casilla a dos vehículos distintos al mismo tiempo.
- **Actualización de Interfaz:** Dado que Swing no es seguro para hilos (*Thread-unsafe*), cualquier cambio en la GUI producido por la ejecución de las garitas se delega al hilo de eventos de Swing mediante SwingUtilities.invokeLater().

## 4. Persistencia de Datos

La persistencia se realiza en archivos de texto plano formateados con el separador pipe (|):

- **residentes.txt:** Formato ID|NOMBRE|CASA|SOCIO.
- **vehiculos.txt:** Formato PLACA|MARCA|MODELO|COLOR|TIPO|ID_RESIDENTE.

**Flujo de persistencia:**

- **Carga (cargarDatos):** Se ejecuta al iniciar la aplicación (VentanaPrincipal), construyendo en memoria las listas de residentes y vehículos.
- **Guardado (guardarDatos):** Se ejecuta mediante un listener WindowAdapter.windowClosing al cerrar la ventana principal del programa.